OBJECT ORIENTED SOFTWARE ENGINEERING

Ms. Archana A

Department of Computer Applications

OBJECT ORIENTED SOFTWARE ENGINEERING

Communication Patterns – Client – Dispatcher Server

Archana A
Department of Computer Applications

PES UNIVERSITY

Name: Client-Dispatcher-Server

- The ***Client-Dispatcher-Server*** design pattern introduces an **intermediate layer between clients** and **servers**, the **dispatcher component.**

- It provides **location transparency** by means of a name service, and hides the details of the establishment of the communication connection between clients and servers.

Client-Dispatcher Server Patterns

Context:

- A software **system integrating a set of distributed servers**, with the servers running locally or distributed over a network.

Problem:

- **Core functionality of the components should be separated from the details of communication mechanisms.**

- Clients need not know where servers are located. This allows you to change the location of the servers dynamically, and provide resilience to network or server failures..

Solution:

- Provide a **dispatcher component** to **act as an intermediate layer** between clients and servers.
- The dispatcher implements a **name service** that allow clients to refer to **servers by names** instead of **physical location transparency.**
- In addition, the dispatcher is responsible for **establishing the communication channel** between a client and a server.

.

Client-Dispatcher Server Patterns

Structure:

Client:

- Perform domain-specific tasks.
- It accesses operations offered by servers in order to carryout its processing task.
- Before sending a request to a server, the client asks the dispatcher for a communication channel.
- Client uses this channel to communicate with the server.

Class Client	***Collaborators***
Responsibility	• Dispatcher • Server
• Implements a system task. • Requests server connections from the dispatcher, • Invokes services of servers,	

Client-Dispatcher Server Patterns

Server:

- Provides a set of operations to clients.
- It either registers itself or is registered with the dispatcher by its name.
- A server component can be located on the same computer as a client, or may be reachable via network.

.

Class Server	***Collaborators*** • Client • Dispatcher
Responsibility • Provides services to clients. • Registers itself with the dispatcher.	

Client-Dispatcher Server Patterns

Dispatcher:

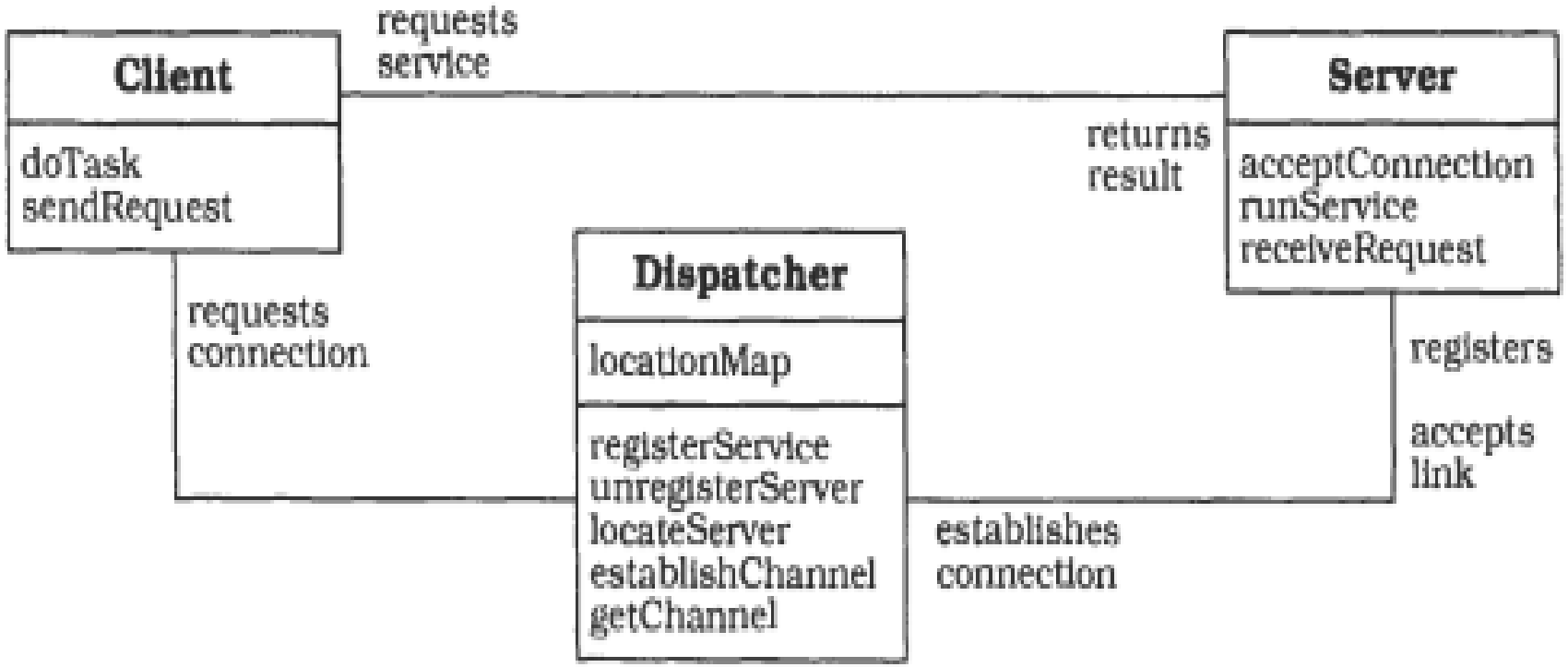
- Offers functionality for establishing communication channels between clients and servers. If the dispatcher cannot initiate a communication link with the requested server, it informs the client about the error it encountered.
- Registers or unregisters servers.
- Maintain a map of server locations.

Class Dispatcher	**Collaborators**
Responsibility • Establishes communication channels between clients and servers. • Locates servers. • (Un-)Registers servers. • Maintains a map of server locations.	• Client • Server

General Structure:

The static relationship between client server and the dispatcher are as follows:

.



Variants:

- Distributed dispatchers:
  - Instead of using a single dispatcher, distributed dispatcher are introduced. Here when a dispatcher receives a client request, it establishes a connection with the dispatcher on the target node.
  - The remote dispatcher initiates a connection with the requested server and sends the communication channel back to the first dispatcher. The channel is then returned to the client.

Known uses:

- Sun's implementation of RPC (Remote Procedure Call) is based upon principles of CDS design pattern

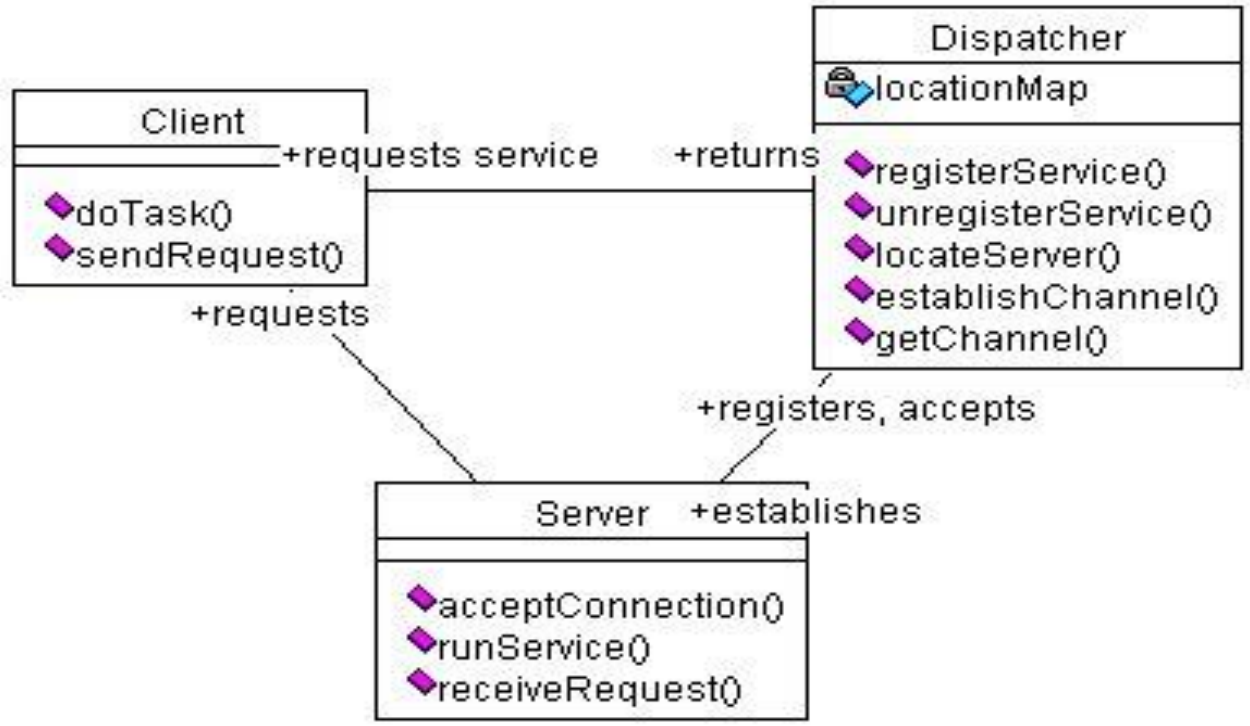
Consequences:

- **Benefits**:
  - Exchangeability of servers.
  - Location and migration transparency
  - Re-configuration
  - Fault tolerance
- **Liabilities**:
  - Lower efficiency through indirection and explicit connection establishment.
  - Sensitivity to change in the interfaces of the dispatcher component.

Working of each components of CDS pattern

General Structure:

.



Working of Client-Dispatcher Server Patterns

Client:

- The task of a **client** is to perform **domain-specific tasks**.
- The **client** accesses operations offered by servers in order to carry out its processing tasks.
- Before sending a request to server, the client asks the dispatcher for a **communication channel**

Server:

- A server provides a set of operations to clients. It either **registers itself** or is **registered with the dispatcher by its name and address**.
- A server component may be located on the same computer as a client or may be reachable via a network.

Working of CDS Pattern…

Dispatcher:

The dispatcher offers functionality for **establishing communication channels** between **clients** **and** **servers**

A typical scenario for the Client-Dispatcher-Server design pattern includes the following phases.

1. **A server registers itself with the dispatcher component.**
2. **At a later time, a client asks the dispatcher for a communication channel to specified server.**
3. **The dispatcher looks up the server that is associated with the name specified by the client in its registry.**

Working of CDS Pattern…

4. The dispatcher establishes a communication link to the server. If it is able to initiate the connection successfully, it returns the communication channel to the client. If not, it sends the client an error message.

5. The client uses the communication channel to send a request directly to the server.

6. After recognizing the incoming request, the server executes the appropriate service.

7. When the service execution is completed, the server sends the results back to the client

Working of CDS Pattern…

Key Benefits of Client-Dispatcher-Server Pattern

- **Decoupling:** Clients do not need to know the location or details of servers.
- **Scalability:** Servers can be added or removed dynamically without affecting clients.
- **Load Balancing:** The dispatcher can distribute requests evenly across servers.
- **Fault Tolerance:** If a server fails, the dispatcher can redirect requests to another available server.
- **Flexibility:** The pattern supports dynamic service discovery and routing.

Application: **1. Online Gaming Platforms**
In online multiplayer games, players (clients) connect to game servers to play together. The game servers may be distributed across different regions to reduce latency.

How Client-Dispatcher-Server is Used:

Client: The player's gaming device (PC, console, or mobile).

Dispatcher: A matchmaking service that assigns players to the most suitable game server based on factors like latency, server load, and player skill level.

Server: The game server hosting the multiplayer session.

Example: how it works?

A player launches a game and requests to join a multiplayer match.

The dispatcher (matchmaking service) identifies the best available server based on the player's location and current server load.

The player is connected to the selected server, and the game begins..

THANK YOU

Ms. Archana A
Department of Computer Applications

archana@pes.edu

+91 80 6666 3333 Extn 392